



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 二叉树的编程与实现

一、实验目的

掌握指针变量、动态变量的含义，掌握二叉树的结构特征，以及各种存储结构的特点及适用范围，掌握指针类型描述、访问和处理二叉树的运算。

二、实验原理

参照课本 p.70-80, Figure4.13-4.25:

二叉树是一种常见的非线性数据结构，每个结点最多有两个孩子结点，分别称为左孩子和右孩子。在链式存储结构中，二叉树的每个结点通常由数据域、左孩子指针域和右孩子指针域组成。其中，数据域用于存放结点信息，左、右指针分别用于指向该结点的左子树和右子树。通过这种指针连接方式，可以有效表示二叉树中结点之间的层次关系。

二叉树可以分为多种类型，其中二叉查找树是一种具有有序性质的特殊二叉树。对于二叉查找树中的任意结点，其左子树中所有结点的关键字值均小于该结点的关键字值，而右子树中所有结点的关键字值均大于该结点的关键字值。因此，二叉查找树中的元素具有一定的排列规律，便于进行查找、插入和删除等操作。若对二叉查找树进行中序遍历，通常可以得到一个按关键字从小到大排列的有序序列。

本实验主要研究的是普通二叉树的建立与层次遍历。层次遍历又称广度优先遍历，其访问顺序是从根结点开始，按照从上到下、从左到右的顺序依次访问各层结点。由于在遍历过程中需要先访问较早发现的结点，再访问后发现的结点，因此需要借助队列这一先进先出的数据结构来实现。

层次遍历的基本过程为：首先将二叉树的根结点入队；然后当队列不为空时，取出队头结点并访问该结点；如果该结点存在左孩子，则将左孩子入队；如果存在右孩子，则将右孩子入队。重复上述过程，直到队列为空，此时所有结点均已按照层次顺序完成访问。由于队列具有先进先出的特点，先入队的上一层结点会先被访问，从而保证二叉树可以按照层次顺序输出。

在本程序中，二叉树采用二叉链表方式存储，每个结点包含数据域、左孩子指针和右孩子指针。程序通过动态申请内存创建结点，并使用字符 * 表示空结点。在二叉树的层次遍历过程中，程序使用队列保存待访问的结点，依次输出每一层的结点数据，从而实现二叉树的层次遍历算法。

三、实验内容

已知以二叉树表作为存储结构，写出按层次顺序遍历二叉树的算法

算法思想：本算法采用一个队列 q，先将二叉树根节点入队列，然后退队列，输出该节点，若它有左子树，便将左子树根节点入队列；若有右子树，便将右子树根节点入队列，直到队列空为止。因为队列的特点是先进先出，从而达到按层次顺序遍历二叉树的目的。

四、实验要求

1) 实现二叉树表的层次遍历算法，并给出应用。

五、实验源程序

```
#include <stdio.h>
```

```

#include <stdlib.h>

#define QUEUE_MAX_SIZE 100

// 二叉树结点结构
typedef struct TreeNode
{
    char data;           // 结点数据
    struct TreeNode *left; // 左孩子指针
    struct TreeNode *right; // 右孩子指针
} TreeNode, *BiTree;

// 队列结构，用来存放二叉树结点指针
typedef struct
{
    BiTree elem[QUEUE_MAX_SIZE]; // 存放结点指针的数组
    int front;                    // 队头位置
    int rear;                     // 队尾位置
} LinkQueue;

// 函数声明
void InitQueue(LinkQueue *Q);
int IsQueueEmpty(LinkQueue Q);
int IsQueueFull(LinkQueue Q);
void EnQueue(LinkQueue *Q, BiTree node);
BiTree DeQueue(LinkQueue *Q);

BiTree CreateNode(char ch);
void CreateBiTreeByLevel(BiTree *T);
void LevelOrderTraverse(BiTree T);
void DestroyTree(BiTree T);

int main()
{
    BiTree T = NULL;

    printf("按层次顺序创建二叉树，输入 * 表示空结点。\\n");

    CreateBiTreeByLevel(&T);

    printf("\\n 二叉树的层次遍历结果为: \\n");
    LevelOrderTraverse(T);

    DestroyTree(T);
    
```

```

        return 0;
    }

// 初始化队列
void InitQueue(LinkQueue *Q)
{
    Q->front = 0;
    Q->rear = 0;
}

// 判断队列是否为空
int IsQueueEmpty(LinkQueue Q)
{
    return Q.front == Q.rear;
}

// 判断队列是否已满
int IsQueueFull(LinkQueue Q)
{
    return (Q.rear + 1) % QUEUE_MAX_SIZE == Q.front;
}

// 入队操作
void EnQueue(LinkQueue *Q, BiTree node)
{
    if (IsQueueFull(*Q))
    {
        printf("队列已满，无法继续入队！\n");
        return;
    }

    Q->elem[Q->rear] = node;
    Q->rear = (Q->rear + 1) % QUEUE_MAX_SIZE;
}

// 出队操作
BiTree DeQueue(LinkQueue *Q)
{
    BiTree temp;

    if (IsQueueEmpty(*Q))
    {
        return NULL;
    }

```

```

    }

    temp = Q->elem[Q->front];
    Q->front = (Q->front + 1) % QUEUE_MAX_SIZE;

    return temp;
}

// 创建一个新的二叉树结点
BiTree CreateNode(char ch)
{
    BiTree node = (BiTree)malloc(sizeof(TreeNode));

    if (node == NULL)
    {
        printf("内存分配失败! \n");
        exit(1);
    }

    node->data = ch;
    node->left = NULL;
    node->right = NULL;

    return node;
}

// 按层次顺序创建二叉树
void CreateBiTreeByLevel(BiTree *T)
{
    char ch;
    LinkQueue Q;
    BiTree current;
    BiTree newNode;

    InitQueue(&Q);

    printf("请输入根结点: ");
    scanf("%c", &ch);

    if (ch == '*')
    {
        *T = NULL;
        return;
    }
}

```

```

// 创建根结点并入队
*T = CreateNode(ch);
EnQueue(&Q, *T);

// 依次为队列中的每个结点输入左孩子和右孩子
while (!IsEmpty(Q))
{
    current = DeQueue(&Q);

    printf("请输入 %c 的左孩子, * 表示空: ", current->data);
    scanf(" %c", &ch);

    if (ch != '*')
    {
        newNode = CreateNode(ch);
        current->left = newNode;
        EnQueue(&Q, newNode);
    }

    printf("请输入 %c 的右孩子, * 表示空: ", current->data);
    scanf(" %c", &ch);

    if (ch != '*')
    {
        newNode = CreateNode(ch);
        current->right = newNode;
        EnQueue(&Q, newNode);
    }
}

// 二叉树的层次遍历, 并按层分别输出
void LevelOrderTraverse(BiTree T)
{
    LinkQueue Q;
    BiTree current;

    if (T == NULL)
    {
        printf("该二叉树为空树。\\n");
        return;
    }
}
    
```

```

InitQueue(&Q);
EnQueue(&Q, T);

int level = 1;          // 当前层数
int currentCount = 1;    // 当前层还未访问的结点数
int nextCount = 0;       // 下一层的结点数

printf("第%d 层: ", level);

while (!IsEmpty(Q))
{
    current = DeQueue(&Q);

    printf("%c ", current->data);
    currentCount--;

    // 如果左孩子不为空，则左孩子入队
    if (current->left != NULL)
    {
        EnQueue(&Q, current->left);
        nextCount++;
    }

    // 如果右孩子不为空，则右孩子入队
    if (current->right != NULL)
    {
        EnQueue(&Q, current->right);
        nextCount++;
    }

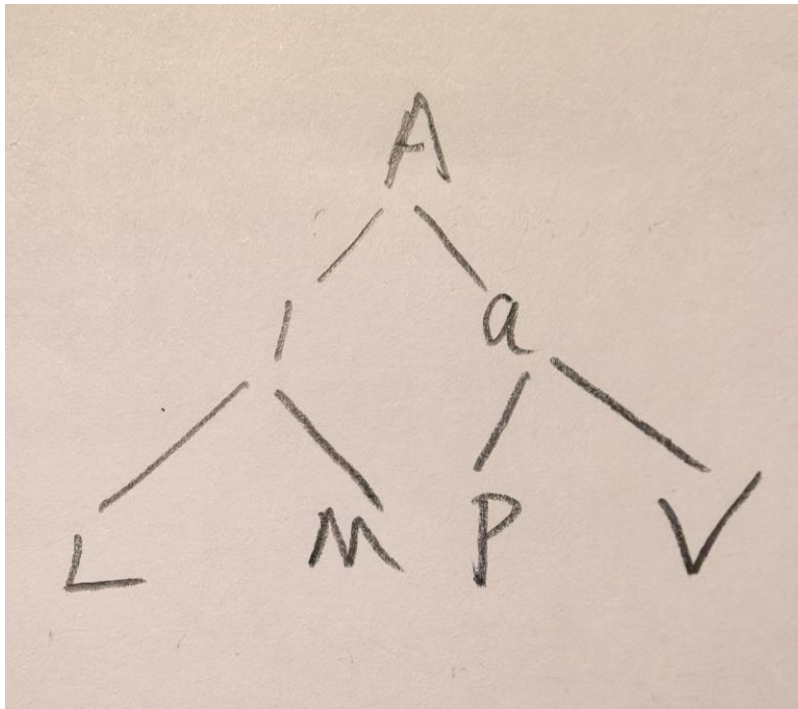
    // 当前层访问完毕，换到下一层
    if (currentCount == 0 && nextCount > 0)
    {
        level++;
        printf("\n 第%d 层: ", level);

        currentCount = nextCount;
        nextCount = 0;
    }
}

printf("\n");
}
    
```

```
// 释放二叉树占用的内存
void DestroyTree(BiTree T)
{
    if (T != NULL)
    {
        DestroyTree(T->left);
        DestroyTree(T->right);
        free(T);
    }
}
```

六、实验结果



按层次顺序创建二叉树，输入 * 表示空结点。

请输入根结点：A

请输入 A 的左孩子，* 表示空：1

请输入 A 的右孩子，* 表示空：a

请输入 1 的左孩子，* 表示空：L

请输入 1 的右孩子，* 表示空：M

请输入 a 的左孩子，* 表示空：P

请输入 a 的右孩子，* 表示空：V

请输入 L 的左孩子，* 表示空：*

请输入 L 的右孩子，* 表示空：*

请输入 M 的左孩子，* 表示空：*

请输入 M 的右孩子，* 表示空：*

请输入 P 的左孩子，* 表示空：*

请输入 P 的右孩子，* 表示空：*

请输入 V 的左孩子，* 表示空：*

请输入 V 的右孩子，* 表示空：*

二叉树的层次遍历结果为：

第1层：A

第2层：1 a

第3层：L M P V

PS D:\Epan\Study\数据结构\源程序> █

本实验选取的二叉树结构如图一所示，程序在终端中的运行结果如图二所示。其中，符号“*”用于表示空结点。由运行结果可以看出，程序能够按照从上到下、从左到右的顺序依次输出二叉树各层的结点元素，说明该程序正确实现了二叉树的层次遍历功能。

七、实验心得

通过本次二叉树层次遍历实验，我进一步理解了二叉树的链式存储结构以及指针在树结构中的作用。二叉树的每个结点不仅需要保存数据本身，还需要通过左孩子指针和右孩子指针分别连接左右子树，因此相比顺序表、栈和队列等线性结构，二叉树的逻辑关系更加复杂。

在程序实现过程中，我认识到层次遍历不能简单地使用递归完成，而需要借助队列来保存待访问的结点。程序先将根结点入队，然后每次取出队头结点并输出，再将其非空的左孩子和右孩子依次入队。由于队列具有先进先出的特点，所以可以保证结点按照从上到下、从左到右的顺序被访问。通过这个过程，我更加直观地理解了队列在二叉树遍历中的应用。

此外，在创建二叉树时，使用“*”表示空结点，可以清楚地描述二叉树中某些位置没有孩子结点的情况。实验中还需要注意输入顺序、空结点判断以及动态内存分配等问题，否则容易导致树结构建立错误或遍历结果不正确。

总的来说，本次实验不仅帮助我掌握了二叉树的基本结构和层次遍历算法，也让我体会到不同数据结构之间可以相互配合使用。队列虽然本身是线性结构，但在二叉树这种非线性结构的遍历中起到了关键作用，这加深了我对数据结构综合应用的理解。

